

code



([https://github.com/jermwatt/mlrefined/blob/gh-](https://github.com/jermwatt/mlrefined/blob/gh-pages/blog_posts/13_Multilayer_perceptrons/13_7_General_steepest_descent.ipynb)
[pages/blog_posts/13_Multilayer_perceptrons/13_7_General_steepest_descent.ipynb](https://github.com/jermwatt/mlrefined/blob/gh-pages/blog_posts/13_Multilayer_perceptrons/13_7_General_steepest_descent.ipynb))

share



([https://www.linkedin.com/sharer/sharer.php?](https://www.linkedin.com/sharer/sharer.php?url=https://github.com/jermwatt/mlrefined/blob/gh-pages/blog_posts/13_Multilayer_perceptrons/13_7_General_steepest_descent.ipynb)
[url=https://github.com/jermwatt/mlrefined/blob/gh-](https://www.linkedin.com/sharer/sharer.php?url=https://github.com/jermwatt/mlrefined/blob/gh-pages/blog_posts/13_Multilayer_perceptrons/13_7_General_steepest_descent.ipynb)
[pages/blog_posts/13_Multilayer_perceptrons/13_7_General_steepest_descent.ipynb](https://www.linkedin.com/sharer/sharer.php?url=https://github.com/jermwatt/mlrefined/blob/gh-pages/blog_posts/13_Multilayer_perceptrons/13_7_General_steepest_descent.ipynb))

► CHAPTER 13: MULTI-LAYER PERCEPTRONS ([HTTPS://JERMWATT.GITHUB.IO/MLREFINED/INDEX.HTML](https://jermwatt.github.io/mlrefined/index.html))

13.7 General steepest descent

In this post we describe several variations of normalized gradient descent, known generally as *steepest descent* algorithms. These variations arise out of a small but fundamental change to the gradient descent setup, a change that has significant impact on the form of a descent direction but importantly, not on the convergence of the methods (steepest descent methods converge under the same general condition as gradient descent, i.e., that the length of the steps eventually diminishes).

In [1]:

13.7.1 Generalized Steepest descent

When we first discussed gradient descent the descent direction given by the negative gradient fell out naturally from a geometric understanding of hyperplanes. Here we begin our discussion of steepest descent algorithms by re-visiting gradient descent and deriving the descent direction using a more rigorous mathematical framework. This framework immediately suggests a way to generalize the gradient descent algorithm under different *norms*, which we then explore.

13.7.1.1 Steepest descent direction in the ℓ_2 norm

When first deriving the gradient descent direction of a multi-input function $g(\mathbf{w})$ at a point $\mathbf{v}$ we began by examining the tangent hyperplane to g at this point

$$h(\mathbf{w}) = g(\mathbf{v}) + \nabla g(\mathbf{v})^T (\mathbf{w} - \mathbf{v}) \quad (1)$$

We then reasoned out that since in general we know that the *ascent* direction on a hyperplane is given by its set of 'slopes' - stored in $\nabla g(\mathbf{v})$ - that intuitively therefore the descent direction is given by the negative gradient as $-\nabla g(\mathbf{v})$, or normalized to unit length as $-\frac{\nabla g(\mathbf{v})}{\|\nabla g(\mathbf{v})\|}$. We often used the latter unit-normalized version since after all we care only about the descent *direction*.

This descent direction can be derived more formally as follows. Note that $\mathbf{d} = \mathbf{w} - \mathbf{v}$ is a general search direction centered at the point $\mathbf{v}$. We want to find the unit-length direction $\mathbf{d}$ that provides the smallest evaluation on the hyperplane, i.e., the one that gives the smallest value $g(\mathbf{v}) + \nabla g(\mathbf{v})^T (\mathbf{w} - \mathbf{v}) = g(\mathbf{v}) + \nabla g(\mathbf{v})^T \mathbf{d}$. Since $\mathbf{d}$ is only present in the second term we only need care about finding the unit length direction $\mathbf{d}$ so that $\nabla g(\mathbf{v})^T \mathbf{d}$ is as small as possible (after all, $g(\mathbf{v})$ is constant).

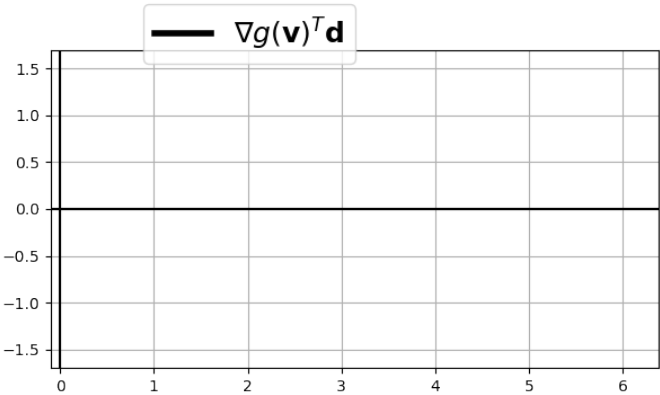
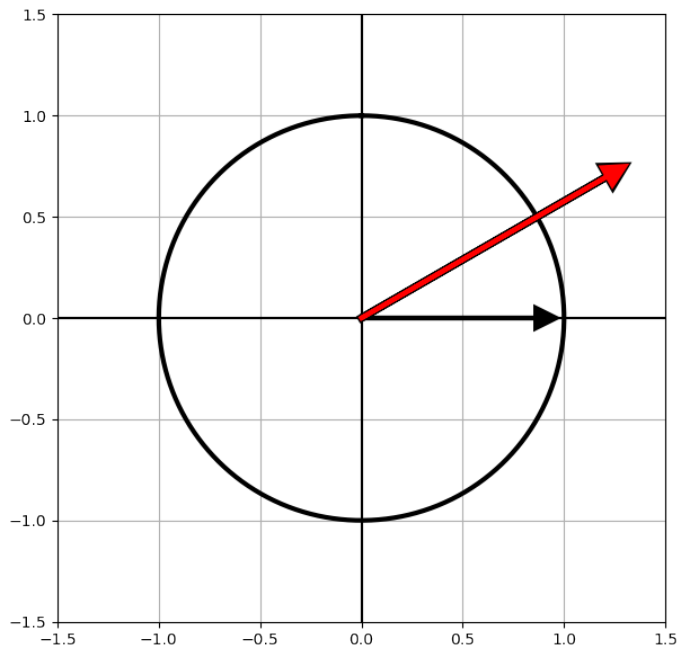
Formally this is a simple constrained minimization problem

$$\begin{aligned} & \underset{\mathbf{d}}{\text{minimize}} && \nabla g(\mathbf{v})^T \mathbf{d} \\ & \text{subject to} && \|\mathbf{d}\|_2 = 1 \end{aligned} \quad (2)$$

Using the toy in the next Python cell we explore possible solutions to this problem in two dimensions for a particular choice of gradient vector, gaining valuable geometric intuition as to what the solution should be. There we plot an example gradient vector $\nabla g(\mathbf{v})$ as a red arrow, along with the ℓ_2 unit ball. Moving the slider from left to right you can test various directions $\mathbf{d}$ (each shown as a black arrow) on this ℓ_2 unit ball computing the inner product $\nabla g(\mathbf{v})^T \mathbf{d}$, whose value is simultaneously plotted in the right panel. As you move the slider right the direction providing the *smallest* value is shown as a green arrow in the left panel, and is highlighted in green in the plot in the right panel.

In [2]:

Out[2]:



•

As can be seen it appears as if indeed $\mathbf{d} = -\frac{\nabla g(\mathbf{v})}{\|\nabla g(\mathbf{v})\|}$ is the direction producing the smallest inner product.

To prove this formally we can use the *inner product rule* which tells us almost immediately what the solution to this problem is. According to the inner product rule $\nabla g(\mathbf{v})^T \mathbf{d}$ can be written as

$$\nabla g(\mathbf{v})^T \mathbf{d} = \|\nabla g(\mathbf{v})\|_2 \|\mathbf{d}\|_2 \cos(\theta)$$

where θ is the angle between $\nabla g(\mathbf{v})$ and $\mathbf{d}$.

Noting that both $\|\nabla g(\mathbf{v})\|_2$ and $\|\mathbf{d}\|_2$ have constant values (the former is the length of the gradient at $\mathbf{v}$, and latter is just 1), the value of $\nabla g(\mathbf{v})^T \mathbf{d}$ is smallest when $\cos(\theta)$ is smallest, i.e., when $\theta = \pi$. In other words, $\mathbf{d}$ must point in the same direction as $-\nabla g(\mathbf{v})$, and also be unit length. Thus we have $\mathbf{d} = -\frac{\nabla g(\mathbf{v})}{\|\nabla g(\mathbf{v})\|_2}$ that is indeed the normalized gradient descent direction. In deriving the gradient descent direction in this way, $\mathbf{d}$ is referred to more generally as the *steepest descent direction*.

13.7.1.2 Steepest descent direction in the ℓ_1 norm

With this setup we can actually choose any norm in the constraint that we wish to determine a descent direction. For example, replacing the ℓ_2 with the ℓ_1 norm we have a similar looking problem

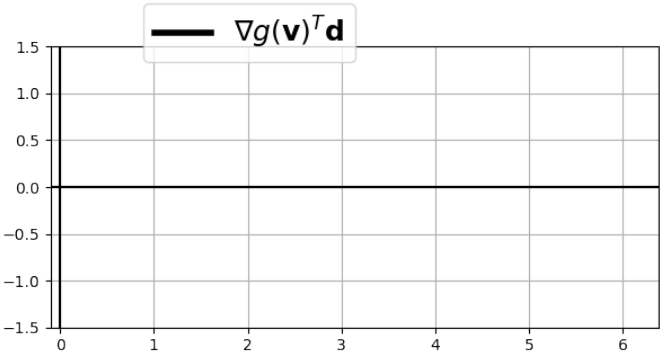
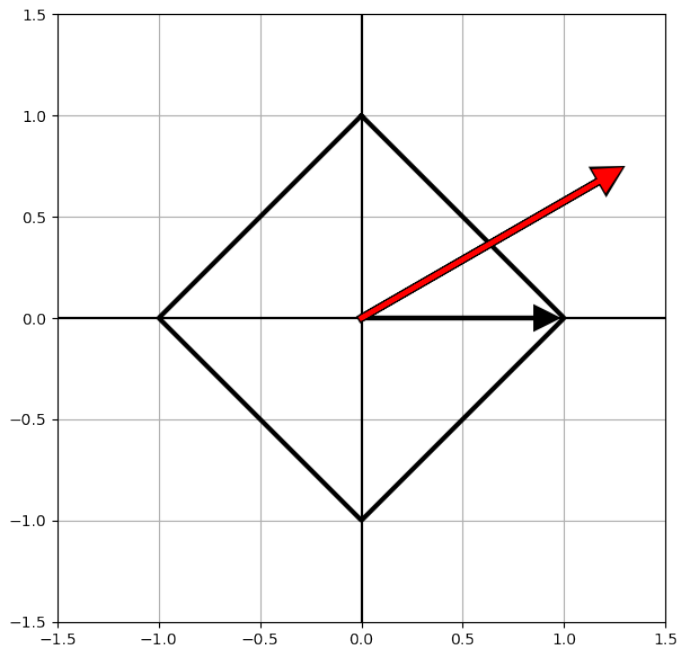
$$\begin{aligned} & \underset{\mathbf{d}}{\text{minimize}} && \nabla g(\mathbf{v})^T \mathbf{d} \\ & \text{subject to} && \|\mathbf{d}\|_1 = 1 \end{aligned} \tag{3}$$

whose solution defines a new steepest descent direction with respect to the ℓ_1 norm. The direction we find here will certainly be different than the ℓ_2 constrained version.

Using the toy in the next Python cell we explore possible solutions to this problem in two dimensions for a particular choice of gradient vector, gaining valuable geometric intuition as to what the solution should be. There we plot an example gradient vector $\nabla g(\mathbf{v})$ as a red arrow, along with the ℓ_1 unit ball. Moving the slider from left to right you can test various directions $\mathbf{d}$ (each shown as a black arrow) on this ℓ_1 unit ball computing the inner product $\nabla g(\mathbf{v})^T \mathbf{d}$, whose value is simultaneously plotted in the right panel. As you move the slider right the direction providing the *smallest* value is shown as a green arrow in the left panel, and is highlighted in green in the plot in the right panel.

In [3]:

Out[3]:

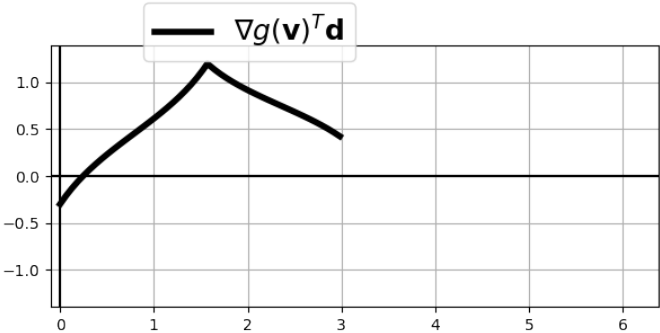
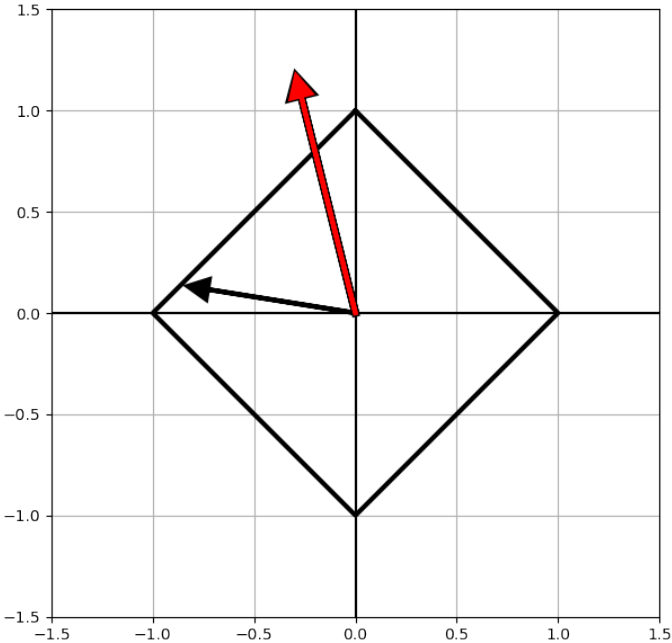


•

Let us try a different a gradient vector $\nabla g(\mathbf{v})$, this time in the second quadrant.

In [4]:

Out[4]:



Toying around with the slider above it looks as if the descent direction here is defined along a *coordinate axis* as the negative direction of only the largest (in magnitude) partial derivative of the gradient. Denoting by j the index of the largest (in magnitude) entry in $\nabla g(\mathbf{v})$

$$j = \operatorname{argmax}_i \left| \frac{\partial}{\partial w_i} g(\mathbf{v}) \right|$$

the descent direction $\mathbf{d}$ will be along the vector

$$\begin{bmatrix} 0 \\ \vdots \\ 0 \\ -\frac{\partial}{\partial w_j} g(\mathbf{v}) \\ 0 \\ \vdots \\ 0 \end{bmatrix} \rightarrow j^{th} \text{ position}$$

which can be written, using the standard basis notation, as $-\frac{\partial}{\partial w_j} g(\mathbf{v}) \mathbf{e}_j$, where $\mathbf{e}_j$ is a standard basis vector with a 1 in its j^{th} position and zeros elsewhere. Dividing this vector by its norm (so that it becomes unit length) we have

$$\mathbf{d} = \frac{-\frac{\partial}{\partial w_j} g(\mathbf{v})}{\left\| -\frac{\partial}{\partial w_j} g(\mathbf{v}) \mathbf{e}_j \right\|_2} \mathbf{e}_j = \frac{-\frac{\partial}{\partial w_j} g(\mathbf{v})}{\left| \frac{\partial}{\partial w_j} g(\mathbf{v}) \right|} \mathbf{e}_j = -\operatorname{sign} \left(\frac{\partial}{\partial w_j} g(\mathbf{v}) \right) \mathbf{e}_j$$

It is certainly possible for more than one partial derivative to be maximal (in magnitude), e.g., when $\nabla g(\mathbf{v}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ in the setup above. In this case not only both $-\operatorname{sign} \left(\frac{\partial}{\partial w_1} g(\mathbf{v}) \right) \mathbf{e}_1$ and $-\operatorname{sign} \left(\frac{\partial}{\partial w_2} g(\mathbf{v}) \right) \mathbf{e}_2$ minimize the inner product, but so will their ℓ_1 length normalized sum $-\frac{\operatorname{sign} \left(\frac{\partial}{\partial w_1} g(\mathbf{v}) \right) \mathbf{e}_1 + \operatorname{sign} \left(\frac{\partial}{\partial w_2} g(\mathbf{v}) \right) \mathbf{e}_2}{2}$.

More generally speaking, if we denote $\mathcal{S}$ the set of indices of those maximal partial derivatives the descent direction that combines all indices in $\mathcal{S}$ is given by

$$\mathbf{d} = -\frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} \operatorname{sign} \left(\frac{\partial}{\partial w_j} g(\mathbf{v}) \right) \mathbf{e}_j \quad (4)$$

This formula can be proven to be true rigorously, as we show in the Appendix Section of this post.

The steepest descent step at $\mathbf{w}^{k-1}$ in this direction looks like

$$\mathbf{w}^k = \mathbf{w}^{k-1} - \alpha \sum_{j \in \mathcal{S}} \text{sign} \left(\frac{\partial}{\partial w_j} g(\mathbf{w}^{k-1}) \right) \mathbf{e}_j \quad (5)$$

where the index set $\mathcal{S}$ changes from step to step depending on the full gradient $\nabla g(\mathbf{w}^{k-1})$, and the constant $\frac{1}{|\mathcal{S}|}$ has been absorbed into α . Given how each direction is formed these steps tend to go along the coordinate directions, like a *coordinate descent* approach. This descent step appears less useful than the ℓ_2 analog in general, since we must still compute the entire gradient in order to use it and then we 'throw away' all but its largest values.

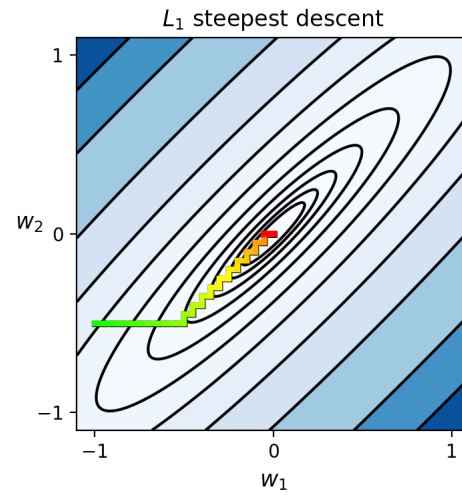
Example 1. ℓ_1 steepest descent applied to minimizing a quadratic

In the next Python cell we use the ℓ_1 steepest descent approach to find the minimum of

$$g(w_1, w_2) = 0.26(w_1^2 + w_2^2) - 0.48w_1w_2 \quad (6)$$

located at the origin. We draw the path taken by this approach directly on the function contours - which are colored green as the method starts and gradually change red as it halts.

In [5]:



13.7.1.3 Steepest descent direction in the ℓ_∞ norm

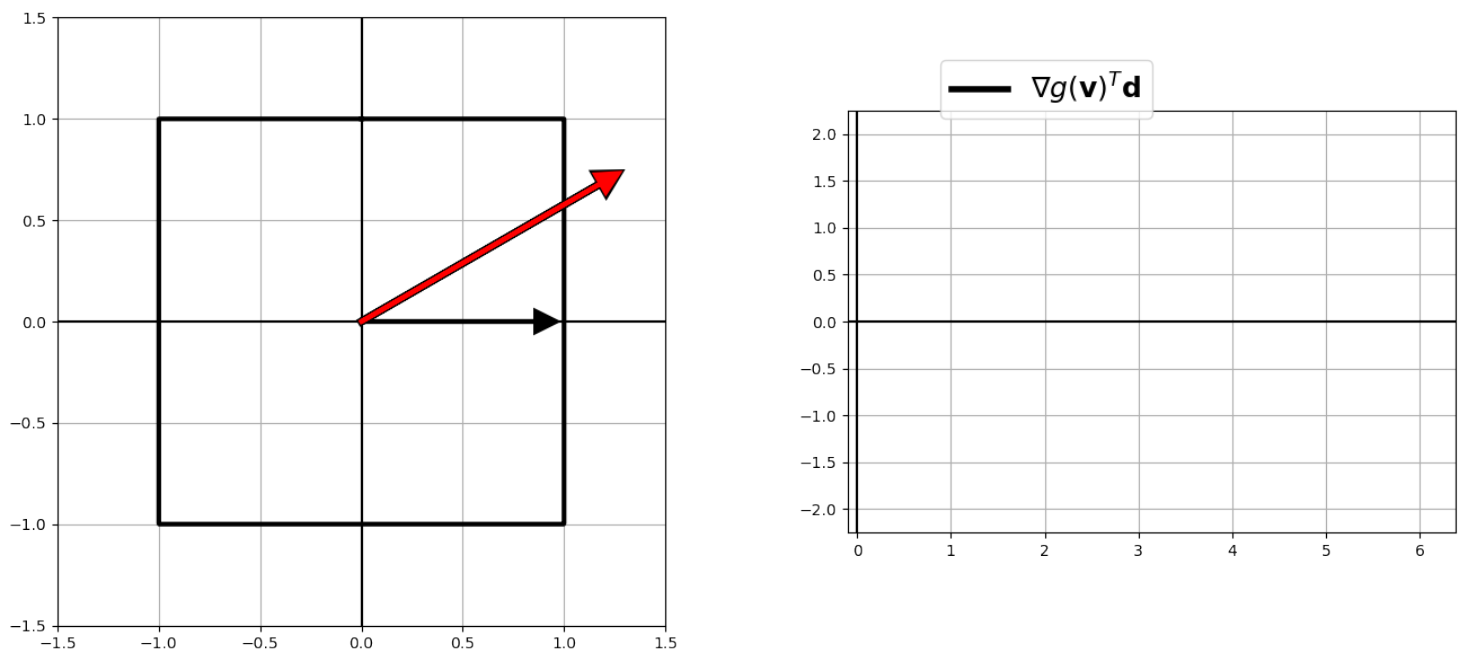
Replacing the ℓ_2 norm with the ℓ_∞ norm in our original constrained optimization problem we have a similar looking problem whose solution defines a new kind of steepest descent direction

$$\begin{aligned} & \underset{\mathbf{d}}{\text{minimize}} && \nabla g(\mathbf{v})^T \mathbf{d} \\ & \text{subject to} && \|\mathbf{d}\|_\infty = 1 \end{aligned} \tag{7}$$

Using the toy in the next Python cell we explore possible solutions to this problem in two dimensions for a particular choice of gradient vector, gaining valuable geometric intuition as to what the solution should be. Here once again we plot an example gradient vector $\nabla g(\mathbf{v})$ as a red arrow, along with the ℓ_∞ unit ball. Moving the slider from left to right you can test various directions $\mathbf{d}$ (each shown as a black arrow) on this ℓ_∞ unit ball computing the inner product $\nabla g(\mathbf{v})^T \mathbf{d}$, whose value is simultaneously plotted in the right panel. As you move the slider right once more the direction providing the *smallest* value is shown as a green arrow in the left panel, and is highlighted in green in the plot in the right panel.

In [6]:

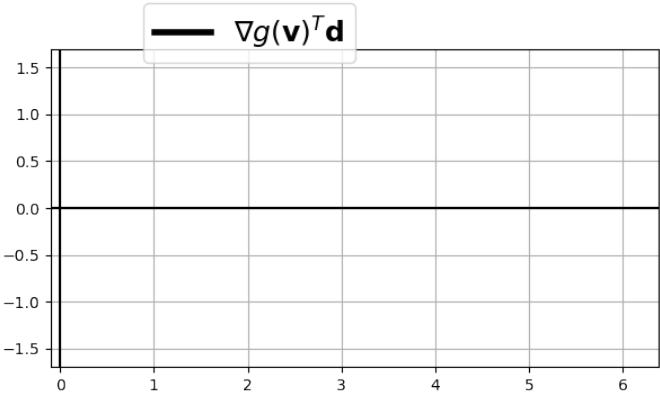
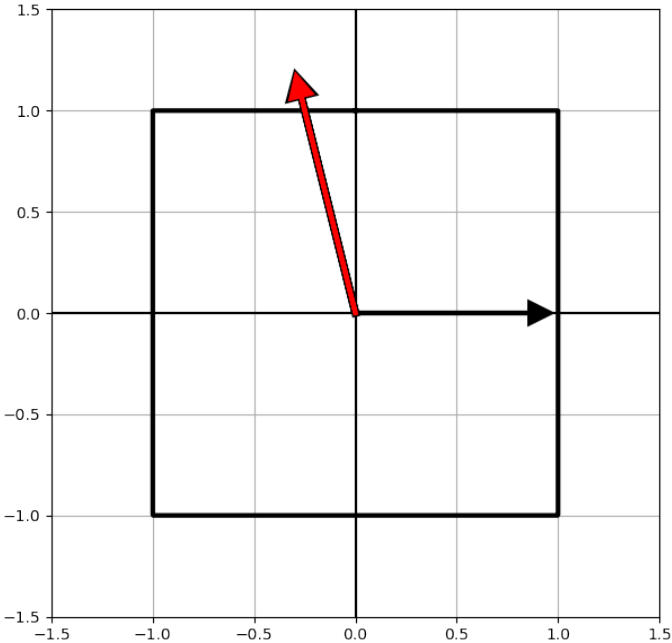
Out[6]:



Based on the example above you may now have a hunch about what **d** should be. But let's make sure by examining a different gradient vector $\nabla g(\mathbf{v})$.

In [7]:

Out[7]:



Toying around with the slider above it looks as if *each* entry of the descent direction here is the negative unit length partial derivative along its respective coordinate. That is, the j^{th} entry is given as

$$d_j = \frac{\frac{\partial}{\partial w_j} g(\mathbf{v})}{\left| \frac{\partial}{\partial w_j} g(\mathbf{v}) \right|} = \text{sign} \left(\frac{\partial}{\partial w_j} g(\mathbf{v}) \right)$$

and hence the entire descent direction can be written simply as

$$\mathbf{d} = -\text{sign}(\nabla g(\mathbf{v})) \quad (8)$$

These directions tend to most often be constrained to the corners of the unit square or - in higher dimensions - the unit hypercube (the exceptions being when one partial derivative is exactly equal to zero).

This intuited formula is in fact correct, and can be proven more rigorously as show in the Appendix Section of this post.

The steepest descent step at $\mathbf{w}^{k-1}$ in this direction looks like

$$\mathbf{w}^k = \mathbf{w}^{k-1} - \alpha \text{sign}(\nabla g(\mathbf{w}^{k-1})) \quad (9)$$

Due to the $\text{sign}(\cdot)$ function these steps tend to be at 'diagonal' traveling largely in directions defined by the corners of the unit square. Because the set of directions is restricted one often sees the use of coordinate-wise steplength parameters with this method (e.g., Rprop (<https://en.wikipedia.org/wiki/Rprop>)).

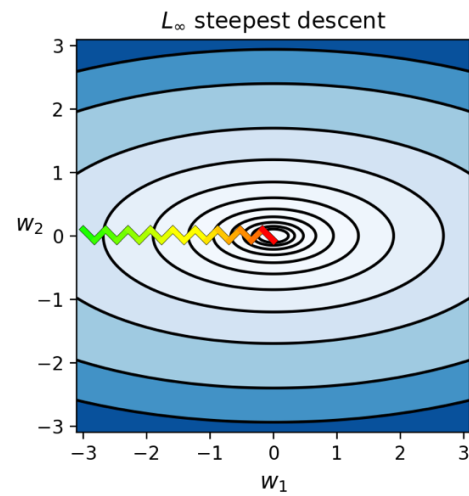
Example 2. ℓ_∞ steepest descent applied to minimizing a quadratic

In the next Python cell we use the ℓ_∞ steepest descent approach to find the minimum of

$$g(w_1, w_2) = w_1^2 + 5w_2^2 \quad (10)$$

located at the origin. We draw the path taken by this approach directly on the function contours - which are colored green as the method starts and gradually change red as it halts.

In [8]:



Appendix

Here we provide formal proofs of the ℓ_1 and ℓ_∞ descent directions. Note that throughout for ease of reading we denote $\mathbf{a} = -\nabla g(\mathbf{v})$.

A.1 Formal derivation of ℓ_1 descent direction

In what follows we find a closed form solution to

$$\begin{aligned} & \underset{\mathbf{d}}{\text{maximize}} && \mathbf{a}^T \mathbf{d} \\ & \text{subject to} && \|\mathbf{d}\|_1 = 1 \end{aligned} \quad (11)$$

written entry-wise as

$$\begin{aligned} & \underset{d_1, d_2, \dots, d_n}{\text{maximize}} && a_1 d_1 + a_2 d_2 + \dots + a_n d_n \\ & \text{subject to} && |d_1| + |d_2| + \dots + |d_n| = 1 \end{aligned} \quad (12)$$

Firstly notice that if the mathematical sign of d_i differs from that of a_i , then replacing d_i with $-d_i$ would increase (or at least not decrease) the objective value without violating the equality constraint. Therefore by writing d_i as

$$d_i = \text{sign}(a_i) y_i$$

where y_i 's are now all nonnegative, our original problem can be reformulated equivalently as

$$\begin{aligned} & \underset{y_1, y_2, \dots, y_n}{\text{maximize}} && a_1 \text{sign}(a_1) y_1 + a_2 \text{sign}(a_2) y_2 + \dots + a_n \text{sign}(a_n) y_n \\ & \text{subject to} && |\text{sign}(a_1) y_1| + |\text{sign}(a_2) y_2| + \dots + |\text{sign}(a_n) y_n| = 1 \\ & && y_1, y_2, \dots, y_n \geq 0 \end{aligned} \quad (13)$$

Noticing that we have $a_i \text{sign}(a_i) y_i = |a_i| y_i$ (in the objective), and $|\text{sign}(a_i) y_i| = |\text{sign}(a_i)| |y_i| = y_i$, we can rewrite the optimization problem as

$$\begin{aligned} & \underset{y_1, y_2, \dots, y_n}{\text{maximize}} && |a_1| y_1 + |a_2| y_2 + \dots + |a_n| y_n \\ & \text{subject to} && y_1 + y_2 + \dots + y_n = 1 \\ & && y_1, y_2, \dots, y_n \geq 0 \end{aligned} \quad (14)$$

Finally, denoting by j an index of the largest entry in $\mathbf{a}$ (in magnitude)

$$j = \underset{i=1, 2, \dots, n}{\text{argmax}} |a_i|$$

we can find an upperbound to the objective

$$|a_1| y_1 + |a_2| y_2 + \dots + |a_n| y_n \leq |a_j| y_1 + |a_j| y_2 + \dots + |a_j| y_n = |a_j|$$

Notice that the j^{th} standard basis vector (having 1 in its j^{th} position and zero elsewhere) gives the same value as this upperbound, and hence $\mathbf{y} = \mathbf{e}_j$ is a solution to the reformulated optimization problem. Moreover if there are several entries of $\mathbf{a}$ that are equally maximal in magnitude the above is true for each of their corresponding indices, the set of which we denote as $\mathcal{S}$.

All together then a solution is given as

$$\mathbf{d}^\star = \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} \text{sgn}(a_j) \mathbf{e}_j$$

where $|\mathcal{S}|$ denotes the number of elements in $\mathcal{S}$.

A.2 Formal derivation of ℓ_∞ descent direction

Here we find a closed form solution to

$$\begin{aligned} & \underset{\mathbf{d}}{\text{maximize}} && \mathbf{a}^T \mathbf{d} \\ & \text{subject to} && \|\mathbf{d}\|_\infty = 1 \end{aligned} \tag{15}$$

written entry-wise as

$$\begin{aligned} & \underset{d_1, d_2, \dots, d_n}{\text{maximize}} && a_1 d_1 + a_2 d_2 + \dots + a_n d_n \\ & \text{subject to} && \max\{|d_1|, |d_2|, \dots, |d_n|\} = 1 \end{aligned} \tag{16}$$

Notice that the constraint set

$$\mathcal{S} = \{\mathbf{d} \mid \max(|d_1|, |d_2|, \dots, |d_n|) = 1\}$$

is a subset of the set

$$\mathcal{T} = \{\mathbf{d} \mid -1 \leq d_1, d_2, \dots, d_n \leq 1\}$$

Therefore the maximum objective value subject to $\mathbf{d} \in \mathcal{S}$ is upperbounded by the maximum objective value subject to $\mathbf{d} \in \mathcal{T}$. The latter problem can be then written as

$$\begin{aligned} & \underset{d_1, d_2, \dots, d_n}{\text{maximize}} && a_1 d_1 + a_2 d_2 + \dots + a_n d_n \\ & \text{subject to} && -1 \leq d_i \leq 1, \forall i \end{aligned} \tag{17}$$

Now note that this problem can be broken down into n independent optimization problems of the form

$$\begin{aligned} & \underset{\mathbf{d}}{\text{maximize}} && \mathbf{a}^T \mathbf{d} \\ & \text{subject to} && -1 \leq d_i \leq 1 \end{aligned} \tag{18}$$

with the easily calculable solution given as $d_i^* = \text{sign}(a_i)$. Taken all together

$$\mathbf{d}^* = \text{sign}(\mathbf{a})$$

maximizes the objective subject to $\mathbf{d} \in \mathcal{T}$. Given we have that $\mathbf{d}^* \in \mathcal{S}$ as well, $\mathbf{d}^*$ also maximizes the objective subject to $\mathbf{d} \in \mathcal{S}$.